DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
BIRLA INSTITUTE OF TECHNOLOGY, MESRA
JAIPUR CAMPUS, JAIPUR

PROJECT-I

AI-Powered Cloud Resource Demand Prediction and Auto-Scaling System

SYSTEM REQUIREMENTS SPECIFICATION (SRS) & SYSTEM DESIGN SPECIFICATION (SDS) REPORT

Role / Designation	Name & Roll Number	Designation / Class
Team Member-1	Name: Pratistha Singh Roll No: Btech/25047/24	Class: CSE (sec A)
Team Member-2	Name: Arushi Goel Roll No: Btech/25111/24	Class: CSE (sec A)
Faculty Mentor	Name: Faculty Mentor	Designation: Assistant Professor

1. Introduction

1.1 Purpose

Moderncloud applications host highly dynamic web services that experience continuously changing and unpredictable user workloads. Traditional reactive auto-scaling models (e.g., standard Kubernetes Horizontal Pod Autoscaler) trigger infrastructure adjustments only after physical metric thresholds (such as CPU utilization exceeding 80%) are breached. Because launching new container replicas or virtual machines incurs a non-negligible initialization lag (ranging from 30 to 180 seconds), reactive systems respond too late during high-velocity burst traffic. This provisioning delay results in transient resource exhaustion, latency spikes, and severe Service Level Agreement (SLA) violations.

This research project aims to design, implement, and validate an intelligent, closed-loop demand prediction and proactive auto-scaling framework for dynamic cloud infrastructure. By replacing standard reactive mechanisms with predictive time-series deep learning architectures (including Gated Recurrent Units [GRU] and Long Short-Term Memory [LSTM] networks), the system anticipates workload spikes 3 to 5 minutes into the future. The scaling engine programmatically communicates with the Kubernetes API to adjust container pod replicas pre-emptively, eliminating cold-start delays, preventing SLA penalties, and optimizing overall operational costs and cluster energy consumption.

1.2 Scope

Theprimaryscope of this project addresses the core research problem of cloud resource management under volatile traffic patterns:

- Infrastructural Volatility Management: Overcoming the physical spin-up and readiness probe delays inherent in microservices container orchestration.
- Predictive Time-Series Pipeline: Developing multi-step forecasting models comparing statistical baselines (ARIMA), machine learning ensembles (Random Forest, XGBoost), and deep sequence models (LSTM, GRU) to forecast CPU utilization, memory demand, and request arrival rates.
- Closed-Loop Kubernetes Control: Building a custom auto-scaling control loop that interacts directly with the Minikube/Kubernetes cluster REST API to execute pre-emptive pod scaling actions.
- Quantifiable Outcomes: Minimizing prediction error (MAE and RMSE), eliminating transient SLA violations under synthetic burst traffic, and achieving a 20% to 30% reduction in cluster compute over-provisioning and energy usage.
- Application Domains: High-traffic cloud e-commerce, microservices orchestration, financial transaction backends, SaaS platforms, and enterprise DevOps pipelines.

1.3 Motivation

Historically,cloudoperators manage volatility using static provisioning or threshold-based reactive auto-scaling. Static provisioning maintains fixed peak capacity, incurring massive financial waste during off-peak periods. Reactive scaling triggers pod instantiation only after overload occurs, creating a 1 to 3 minute vulnerability window where end users suffer request timeouts and degraded performance.

Anticipating demand 3 to 5 minutes in advance transforms cloud resource management from a reactive firefighting mechanism into a proactive optimization process, ensuring container pods reach full readiness exactly as load surges arrive.

1.4 Technology and Tools Used

Category	Technology / Framework	Role & Description in System
Programming Language	Python 3.12	Core pipeline implementation, ML/DL models, and API scaling controller.
ML / DL Frameworks	TensorFlow, PyTorch, Scikit-learn	Deep sequence modeling (LSTM, GRU), ARIMA, XGBoost, and evaluation.
Data Processing & Viz	Pandas, NumPy, Matplotlib	Telemetry normalization, sliding window generation, and performance plotting.
Container Orchestration	Docker, Kubernetes / Minikube	Application microservice containerization and local cluster orchestration.
Observability & Metrics	Prometheus, Grafana	Metric scraping (5s interval), time-series storage, and real-time dashboarding.
Workload & Traffic Gen	Flask / FastAPI, Locust, Apache Bench	Mock web microservices and synthetic normal/burst traffic spike generation.
Public Workload Datasets	Google Cluster Traces, NASA HTTP Logs	Benchmark public workload traces for model training and trace validation.
Automation & CI/CD	Git, GitHub Actions	Version control, automated testing, and CI/CD deployment workflow.

2. Literature Review

2.1 Summary of Existing Work

Classical cloud workload forecasting relied predominantly on linear statistical models such as Auto-Regressive Integrated Moving Average (ARIMA) and Holt-Winters Exponential Smoothing. While effective for periodic, stationary load patterns, these models struggle with highly dynamic, non-linear cloud workloads. Tree-based ensemble learning models (Random Forest, XGBoost) improved non-linear feature handling by incorporating lag metrics. Recently, recurrent deep learning architectures (LSTM and GRU) demonstrated state-of-the-art capability in learning long-range temporal dependencies and non-stationary traffic spikes.

2.2 Gaps Identified in Previous Studies

Despite predictive model advancements, critical operational gaps persist in existing literature:

1. Decoupled Prediction and Execution: Most research evaluates forecasting models offline without integrating predictions into live container orchestrators.
2. Provisioning Lag Unaddressed: Traditional autoscalers ignore the 30–180s container cold-start delay required for pod initialization and readiness checks.
3. High Inference Overhead: Heavy deep neural networks can introduce decision latency that compromises real-time control loops.
4. Lack of Unified Telemetry Validation: Few frameworks offer side-by-side observability correlating predicted demand, actual metrics, and live replica counts.

2.3 How Your Research Differs / Improves

This research closes the loop by connecting a lightweight, low-latency deep learning forecasting engine directly to the Kubernetes API. Operating on a 30-second control cycle with a 3–5 minute forward lookahead, the system scales pods pre-emptively, eliminating cold-start delays and validating performance via live Prometheus metrics and Grafana dashboards under real-time synthetic traffic surges.

3. Research Requirements (SRS Part)

3.1 Functional Requirements

Req ID		Detailed Functional Description
FR-01	Data Collection & Preprocessing	Continuously ingest CPU, RAM, and request telemetry from Google/NASA logs and Prometheus. Perform MinMaxScaler normalization, missing value imputation, and sliding window sequence generation (t-12 to t).
FR-02	Model Development & Selection	Train and benchmark ARIMA, Random Forest, XGBoost, LSTM, and GRU models. Automatically select the optimal model checkpoint exhibiting lowest MAE/RMSE on validation sets.
FR-03	Training & Validation Pipeline	Partition time-series data chronologically (70% train, 15% val, 15% test) with rolling-window cross-validation to prevent temporal data leakage.
FR-04	Evaluation & Metrics Module	Compute statistical regression error metrics (MAE, RMSE, MAPE) and threshold breach confusion matrices across candidate models.
FR-05	Proactive Replica Controller	Execute a 30-second control loop consuming 3-5 min forecasted demand, compute required pod replicas, and

		call Kubernetes REST APIs to scale pre-emptively.
FR-06	Telemetry Observability	Scrape cluster telemetry via Prometheus exporters and render multi-track Grafana dashboards comparing actual usage, predicted load, and active replicas.

3.2 Non-Functional Requirements

Category	Specification	Target Benchmark / Constraint
Performance	Model Inference Latency	< 200 ms per evaluation cycle to ensure smooth 30s control loop responsiveness.
Scalability	Workload & Data Capacity	Process 10M+ time-series datapoints across 100+ containerized microservices.
Reproducibility	Pipeline Determinism	100% reproducible training scripts, locked random seeds, and stored scaler parameters.
Constraints	Cluster Resources	Operate within Minikube constraints (min 4 vCPU cores, 8GB RAM, optional CUDA GPU).

3.3 Assumptions and Constraints

1. Dataset Integrity: Benchmarkpublic logs (Google Cluster Traces, NASA HTTP logs) accurately represent real-world cloud burstiness.
2. Cluster Connectivity: The scaling controller maintains reliable low-latency REST API connectivity with the Kubernetes master node.
3. Cold Start Horizon: Microservice pod initialization and readiness probe completion take between 30 and 180 seconds.
4. Resource Headroom: Cluster physical nodes possess sufficient memory and CPU capacity to handle peak pod replica upper bounds.

3.4 Use Case Specifications & Use Case Diagram

The interaction betweenexternal actors (DevOps Engineer, Kubernetes API, Prometheus Exporter) and the system use cases is illustrated in the Use Case Diagram below:

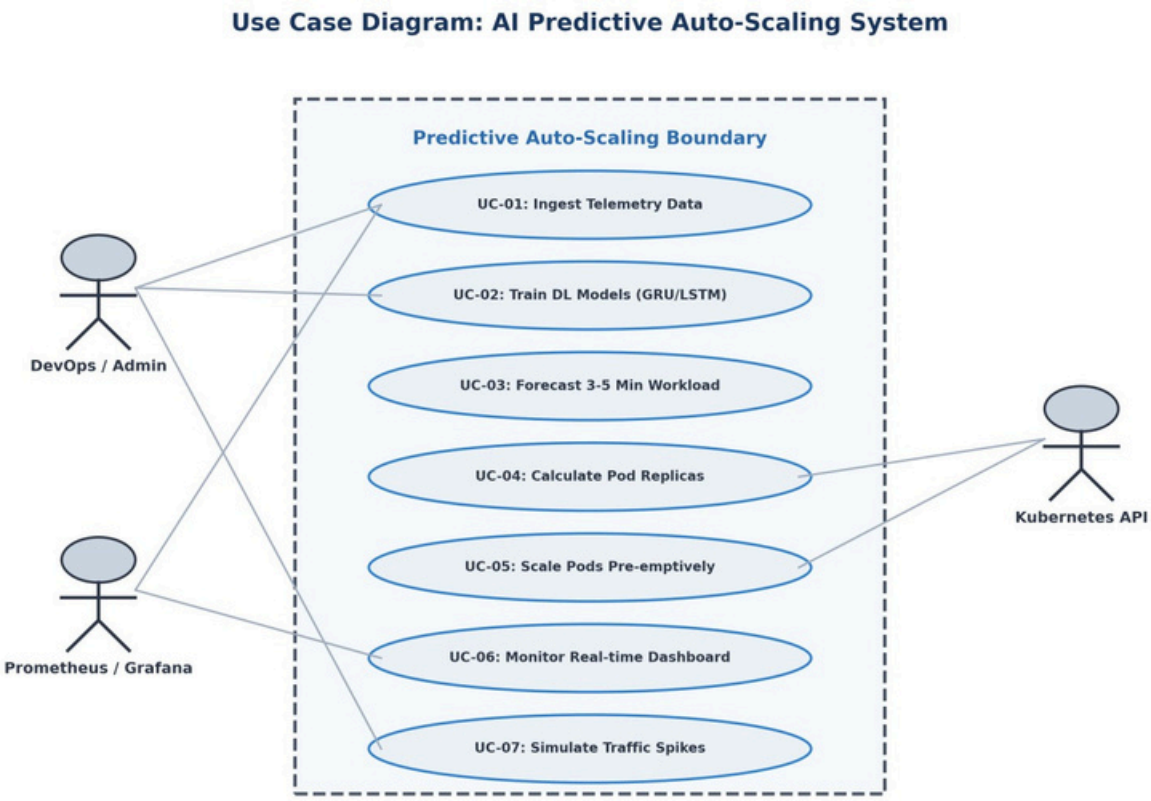


Figure 3.1: Use Case Diagram for AI-Powered Predictive Auto-Scaling System

UC ID	Use Case Title	Primary Actor	Pre-conditions & Flow Summary
UC-01	Ingest Telemetry Data	Prometheus Exporter	Prometheus scrapes metrics every 5s; pipeline standardizes data into sliding window sequences.
UC-02	Train DL Models	DevOps Engineer	Historical traces loaded; pipeline trains GRU/LSTM models and outputs candidate model weights.
UC-03	Forecast 3-5 Min Demand	Prediction Engine	Active model receives sliding window input (t-12 to t) and outputs forecasted load y_hat_{t+k}.
UC-04	Calculate Pod Replicas	Scaling Controller	Consumes forecast y_hat; applies ceiling target formula to compute target replica count N_target.
UC-05	Scale Pods Pre-emptively	Kubernetes API	Controller sends REST PATCH request to K8s API server to scale deployment

			replicas before spike arrives.
UC-06	Monitor Real-time Dashboard	DevOps Engineer	Grafana queries Prometheus TSDB to display side-by-side real load, predicted load, and pod counts.
UC-07	Simulate Traffic Spikes	Locust Generator	Load test script fires synthetic burst traffic at microservice endpoint to evaluate autoscaling performance.

4. System Design (SDS Part)

4.1 Research Workflow & System Architecture

The complete systemoperatesasanend-to-endmulti-stage pipeline connecting workload ingestion, deep learning forecasting, closed-loop scaling control, and live cluster orchestration:

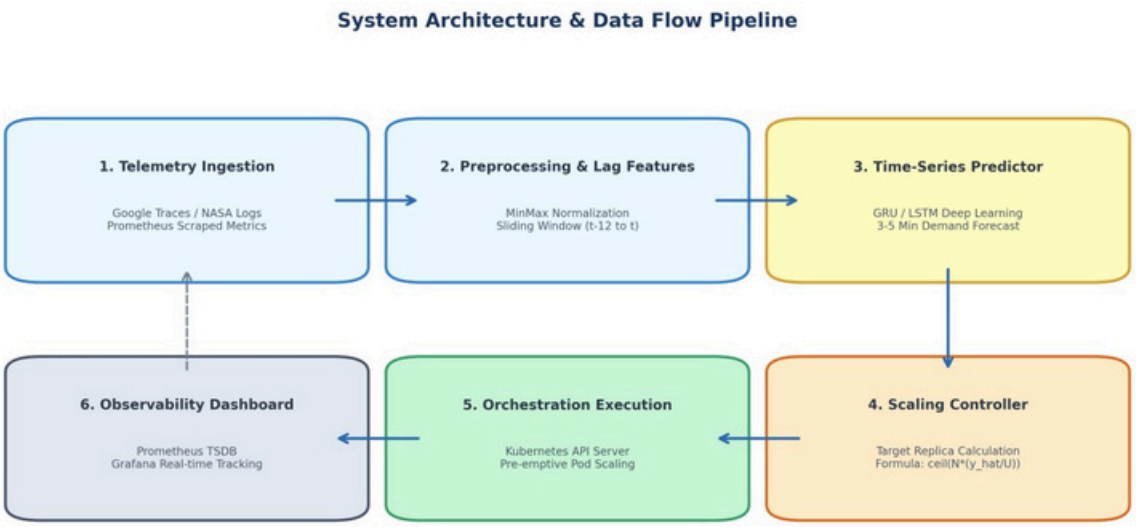


Figure 4.1: End-to-End Predictive Auto-Scaling Architecture & Workflow Pipeline

4.2 Module Design

- 1. Data Preprocessing Module: Cleans raw telemetry traces, imputes missing values, applies MinMaxScaler normalization, and structures data into sliding window sequence vectors (t-t_win to t).
- 2. Feature Engineering Module: Generates historical lag features (CPU_t-1 to CPU_t-10), rolling statistical aggregations (mean, std dev, min, max), derivative load velocities, and calendar temporal indices.
- 3. Model Development & Prediction Module: Houses candidate machine learning (ARIMA, XGBoost) and deep sequence models (LSTM, GRU). Executes forward-pass inferences to generate 3-5 minute ahead workload forecasts.

  4. Evaluation Module: Calculates regression metrics (MAE, RMSE, MAPE) during offline validation and online tracking.

  5. Scaling Decision & Control Engine Module: Consumes forecasted load, executes target replica sizing formulas, and issues pre-emptive scaling calls to the Kubernetes REST API.

  6. Observability & Dashboard Module: Prometheus scrapes live cluster telemetry and Grafana renders multi-track time-series visualization charts.

4.3 Algorithm and Model Design

Thepredictive coreutilizes Gated Recurrent Units (GRU) to capture sequence temporal dynamics efficiently. Mathematical equations governing GRU cell updates:

- Update Gate: $z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$
- Reset Gate: $r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$
- Candidate Hidden State: $\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t] + b)$
- Final Hidden State: $h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$

The Scaling Decision Controller converts the forecasted metric $y_{\hat{}}\{t+k\}$ (where k = 3–5 min lookahead) into discrete pod replicas N_{target} using the formula:

■ TARGET REPLICA DECISION FORMULA

$$N_{target} = \max(N_{min}, \min(N_{max}, \lceil N_{current} \times (y_{\hat{}}\{t+k\} / U_{target}) \rceil))$$

Where:
- $y_{\hat{}}\{t+k\}$ = Forecasted resource utilization (3-5 minutes ahead)
- U_{target} = Target resource utilization threshold (e.g., 70% or 0.70)
- $N_{current}$ = Current active pod replica count
- N_{min}, N_{max} = Safe operational bounds for deployment replica scaling

■ PSEUDO-CODE: CLOSED-LOOP PREDICTIVE AUTO-SCALER CONTROL ENGINE

```
ALGORITHM: ClosedLoopPredictiveAutoScaler
INPUT: Metrics Stream S, Forecast Horizon k=5m, Cycle Interval T=30s, Target Util
U_target=0.70
OUTPUT: Scaling REST API Calls to Kubernetes Cluster

1. WHILE System_Active DO:
2.     x_window <- IngestLastTimesteps(S, lookback=12) # Extract past 60s telemetry
3.     x_norm   <- MinMaxScaler.transform(x_window)
4.     y_hat    <- GRU_Model.predict(x_norm)            # Forecast demand at t+k
5.     N_curr   <- K8s_API.GetDeploymentReplicas('web-app')
6.
7.     # Calculate Target Replicas
8.     N_raw    <- math.ceil(N_curr * (y_hat / U_target))
9.     N_target <- max(N_min, min(N_max, N_raw))
10.
11.    IF N_target != N_curr THEN:
12.        K8s_API.PatchDeploymentReplicas('web-app', N_target)
```

```
        LogScalingEvent(Timestamp, y_hat, N_curr, N_target)
    END IF
    Sleep(T=30 seconds)
END WHILE
```

4.4 Database and Storage Design

• RawWorkloadTraceStorage:Historical Google Cluster Traces and NASA HTTP logs stored in compressed Parquet/CSV formats on local disk for batch model training.
• Prometheus Time-Series Database (TSDB): In-memory and disk-backed TSDB retaining scraped cluster metrics (CPU, memory, request latency) at 5-second sampling rates with a 15-day retention policy.
• Model Checkpoint Storage: Storing trained model weights (.h5 / .pt format), scaler objects (scaler.pkl), and hyperparameter experiment logs.

4.5 Experimental Design & Setup

Parameter / Dimension	Experimental Setup & Configuration Details
Data Partitioning Split	70% Training | 15% Validation | 15% Testing (Chronological Split)
Lookback Window (t_win)	12 to 30 past timesteps (60 to 150 seconds historical context)
Forecasting Horizon (k)	3 to 5 minutes lookahead (6 to 10 future timesteps)
Deep Learning Hyperparameters	Optimizer: Adam (lr=0.001), Loss: MSE, Epochs: 100, Batch Size: 64, Hidden Units: 64/128
Hardware & Cluster Setup	Minikube v1.32, 4 vCPU cores, 8GB RAM, Docker runtime, NVIDIA GPU acceleration

7. References

[1] GoogleInc., 'Google Cluster Workload Traces v2,' Google Cloud Platform Datasets, 2020. [Online]. Available: https://github.com/google/cluster-data.

[2] IEEE Xplore Digital library research paper reference guide

[3] S. Hochreiter and J. Schmidhuber, 'Long Short-Term Memory,' Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] K. Cho et al., 'Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation,' in Proc. EMNLP, 2014, pp. 1724–1734.

[5] Kubernetes Authors, 'Horizontal Pod Autoscaling Architecture and Specifications,' Kubernetes Documentation, 2024. [Online]. Available: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/.

[6] Prometheus Authors, 'Prometheus: An Open-Source Monitoring and Alerting Toolkit,' Cloud Native Computing Foundation (CNCF), 2024.

[7] Grafana Labs, 'Grafana Observability and Dashboarding Platform Documentation,' Grafana Labs, 2024.

[8] T. Chen and C. Guestrin, 'XGBoost: A Scalable Tree Boosting System,' in Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.